

Roteiro — Aula 4: VPS, Docker, Kamal e deploy

Deck: `slides/build/aula-04-vps-docker-kamal-e-deploy.pptx` (61 slides) **Apostila da turma:** `04-deploy.md` · **Checkpoint:** `aula-04`

A aula mais arriscada das quatro. O gargalo não é digitar, é clicar em portal — e portal falha. Leia a seção seguinte antes de qualquer coisa.

O que preparar com antecedência

Estas quatro coisas **não ensinam nada ao serem feitas doze vezes**, e devolvem ~40 minutos:

#	O que	Quando
1	Registros DNS <code>aluno1.capacita...</code> <code>alunoN.capacita</code> já criados no seu Cloudflare, apontando para IP <code>0.0.0.0</code> (eles trocam pelo IP da VM deles)	véspera
2	Um certificado Origin CA wildcard <code>*.capacita.<seu-domínio></code> , com o <code>.pem</code> e a chave prontos para distribuir	véspera
3	Verificar que todo mundo já tem crédito Azure aparecendo no portal	uma semana antes
4	Uma conta Resend com o domínio verificado, e uma API key para a turma	véspera

E o mais importante:

Faça o percurso inteiro sozinho uma vez, com uma VM descartável, antes do encontro. É a única forma de descobrir onde a sua assinatura vai reclamar de cota. Se você não fizer isso, a aula vai parar num erro que você está vendo pela primeira vez, na frente de todo mundo.

Antes de começar

- [] Portal Azure aberto e logado, numa aba anônima (para não vazar recursos do SEEM na projeção).
- [] Cloudflare aberto no domínio da capacitação.
- [] GitHub do repositório da capacitação aberto em Settings → Secrets and variables.
- [] Sua VM de teste **destruída**, para você fazer o percurso junto com eles do zero.
- [] `docs/troubleshooting.md` aberto numa aba — você vai usar.
- [] Certificado Origin CA e a lista de subdomínios prontos para colar no chat.

Cronograma

Relógio	Slides	Bloco	Min
00:00	1–7	VPS, comparação, por que não Kubernetes	16
00:16	8–14	Criar a VM (mão na massa)	35
00:51	15–21	Ubuntu: Docker, permissões, swap, SSH	32
01:23	—	Intervalo	10
01:33	22–26	Docker: imagem, Dockerfile, registry	16
01:49	27–35	Kamal: deploy.yml, accessory, segredos	26
02:15	36–44	CI/CD e OIDC — e configurar o OIDC	45
03:00	45–52	TLS, certificado, CA, Cloudflare	22
03:22	53–56	Primeiro deploy e operação	30
03:52	57–61	Prática, recapitulação, fim	10

Dá 4h02, e é otimista — assume que ninguém pega `NotAvailableForSubscription` e que o *subject* do OIDC acerta de primeira. Nenhuma das duas costuma acontecer.

Plano B, decidido de véspera

Escolha **agora** qual você vai usar:

B1 — Cortar o OIDC. Em vez do pipeline pelo Actions, cada aluno roda o Kamal da própria máquina:

```
GHCR_USER=... AZURE_VM_IP=... APP_HOST=... bundle exec kamal setup
```

Você explica o OIDC pelos slides (40–44, ~12 min) e mostra o workflow rodando **no seu** repositório.

Devolve 45 min e a aula fecha em 3h15. Eles fazem o pipeline em casa, com a apostila.

B2 — Virar demo a partir do intervalo. Se às 01:23 não houver pelo menos metade da turma com VM respondendo ao SSH, pare de esperar. Projete o seu deploy do começo ao fim e distribua o roteiro. **Melhor todo mundo ver funcionando do que metade travar no NSG.**

Bloco a bloco

00:00 — VPS (1–7)

Comece pela comparação (5) e vá para a decisão (6). A frase que resume a aula:

Escolher a ferramenta grande antes do problema grande é a forma mais cara de errar.

E o slide 7 é o que dá credibilidade: admita o custo. Uma VM é ponto único de falha, volume não é backup, o Postgres não é gerenciado.

00:16 — A VM (8-14)

Faça junto, passo a passo, projetado.

- **9** — cotas. Mostre a tela de Uso + quotas antes de tentar criar. Se alguém pegar `NotAvailableForSubscription`, é aqui que se resolve, não depois.
- **11** — **pare neste slide.** "Portas públicas: Nenhuma" é a escolha mais importante da tela. Diga por quê: o assistente, se deixado, libera SSH para a internet inteira, e bots varrem a porta 22 do IPv4 todo, o dia todo.
- **12-13** — chave pública e privada. É o conceito que também explica o JWT da Aula 3 e o TLS que vem daqui a pouco. Não pule.
- **14** — o `chmod 600`. Diga que o SSH **recusa** usar chave com permissão frouxa, e que é a primeira coisa a conferir quando der `Permission denied (publickey)`.

Ponto de decisão às 00:51: se menos da metade conectou por SSH, aplique o plano B2.

00:51 — Ubuntu (15-21)

- **17** — root e permissões. Rápido, mas não pule: a Aula 4 inteira depende disso.
- **18** — variável de ambiente. É o conceito que sustenta o `env.clear` / `env.secret` do Kamal.
- **20** — swap. Explique o OOM killer: numa VM de 1 GiB, Rails + Postgres + build estouram a RAM e o kernel mata o processo que estiver na frente, com uma mensagem que não explica nada.
- **21** — endurecer o SSH. **Fale isto em voz alta:** *"mantenham a sessão atual aberta e testem em outro terminal. Errar a config do SSH com uma sessão só é o jeito clássico de perder a máquina."*

01:33 — Docker (22-26)

Conceitual e rápido. Imagem é receita, container é bolo.

No **25**, as duas decisões: multi-stage (compilador não vai para produção) e usuário não-root (*"é uma linha que muda o tamanho do estrago"*).

01:49 — Kamal (27-35)

Abra o `config/deploy.yml` projetado e percorra junto com os slides.

O slide 31 (`clear` × `secret`) tem o argumento concreto: `JWT_SECRET` no `clear` apareceria em `docker inspect` e no histórico do shell. Se der, mostre um `docker inspect` de verdade.

O **34** (três camadas de segredo) é o slide que amarra tudo. Deixe na tela enquanto explica.

02:15 — CI/CD e OIDC (36–44)

O bloco mais perigoso do dia.

Explique a sequência (39) **antes** de mexer no portal, para eles saberem o que estão construindo: abre a 22 só para o IP do runner, faz o deploy, e fecha — inclusive se falhar.

Depois configurem juntos. **Avise antes**: o *subject* da credencial federada erra na primeira tentativa quase sempre.

```
repo:SEU-USUARIO/SEU-REPO:ref:refs/heads/main
```

Se aparecer `AADSTS700213`, é isso: confira maiúsculas, nome do repositório e branch, caractere por caractere.

E a atribuição de função vai **no NSG**, não na assinatura (44). Insista: se a credencial vazar, o estrago se limita a regras de firewall daquela máquina.

03:00 — TLS (45–52)

- **46–48** — HTTPS é HTTP num túnel; handshake em três passos; certificado e cadeia de confiança.
- **51** — os três modos. A frase que fixa: "com `Flexible`, o cadeado aparece e a senha do usuário trafega em claro no último trecho. É pior que não ter HTTPS, porque mente."
- **52** — por que não Let's Encrypt: com o DNS proxied, o desafio HTTP-01 não chega.

Aqui você distribui o certificado Origin CA wildcard que preparou.

03:22 — O primeiro deploy (53–56)

```
GHCR_USER=... AZURE_VM_IP=... APP_HOST=... bundle exec kamal config
```

Valida sem tocar em nada. **Faça isso antes** — pega variável faltando de graça.

Depois: **Actions** → **Deploy (Kamal)** → **Run workflow** → **command**: `setup`.

O `setup` demora. Use os ~8 minutos para o bloco de backup (56) e para responder pergunta.

E então, o momento da aula:

```
curl https://seunome.capacita.<domínio>/api/v1/status
```

03:52 — Fecho (57–61)

O slide 60 (`O QUE O SEEM-BACKEND TEM A MAIS`) é o convite: "nada disso é difícil depois do que vocês viram hoje. O código está lá, e agora vocês conseguem ler."

Perguntas que vão aparecer

Pergunta	Resposta curta
"Por que não Heroku/Render/Vercel? É mais fácil."	É mesmo, e para muita coisa é a escolha certa. Aqui o objetivo é vocês entenderem o que essas plataformas fazem por vocês — e o custo delas cresce rápido.
"E se a VM cair?"	Cai o sistema. É o custo declarado desta arquitetura. Com backup, você sobe outra em 20 min.
"Por que o banco não é gerenciado?"	Custo. Um Postgres gerenciado na Azure custa mais que a VM inteira. Para 500 usuários numa semana, não se paga.
"Preciso pagar pelo domínio?"	Nesta capacitação não — vocês usam um subdomínio meu. Fora daqui, ~R\$15/ano, ou grátis pelo GitHub Student Pack.
"O crédito da Azure vai acabar?"	São US\$100. Uma B1s ligada 24h custa ~US\$8/mês. Dá quase um ano. Desligue a VM quando não estiver usando.
"Por que abrir e fechar a porta 22 a cada deploy?"	Porque o IP do runner muda a cada execução, e deixar a 22 aberta para a internet é o que os bots procuram.
"Docker Compose não resolveria?"	Resolve subir os containers, sim. Não resolve build remoto, zero-downtime, rollback nem gestão de segredo. É o que o Kamal adiciona.
"Posso usar isso num projeto meu?"	Pode, e é a intenção. Troque o nome do serviço, o domínio e os secrets. O resto é igual.

Depois da capacitação

Mande no grupo:

1. **Desliguem a VM** quando não estiverem usando (`Parar` no portal — parada, ela não consome crédito de computação).
2. O link do `seem-backend`, com o convite do slide 60.
3. `docs/troubleshooting.md` — é o que eles vão abrir quando algo quebrar sem você por perto.